

1 Markovian Drone

1.1 Optimal Value Heatmap Plot

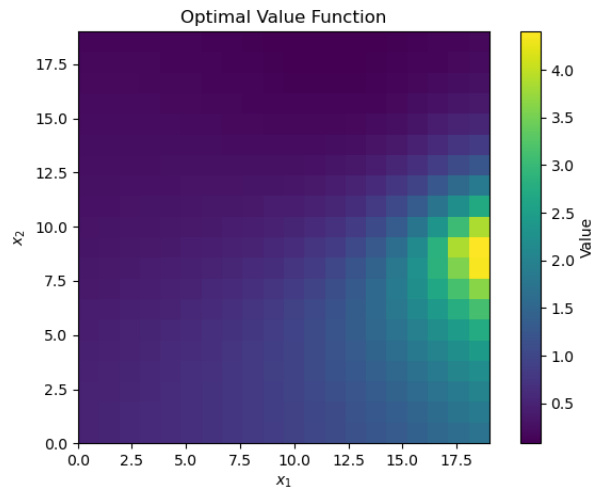


Figure 1: Optimal Value Heatmap Plot

1.2 Optimal Policy Heatmap Plot

As seen in Fig. 2, the simulated drone is following actions depending on the ego-state, denoted by the heatmap. Sometimes the policy is overridden by the storm, that is the reason why the drone is taking sometimes different actions than those of the policy. Also, the simulation runs for 100 steps, so even after reaching the goal, the drone may circle around it.

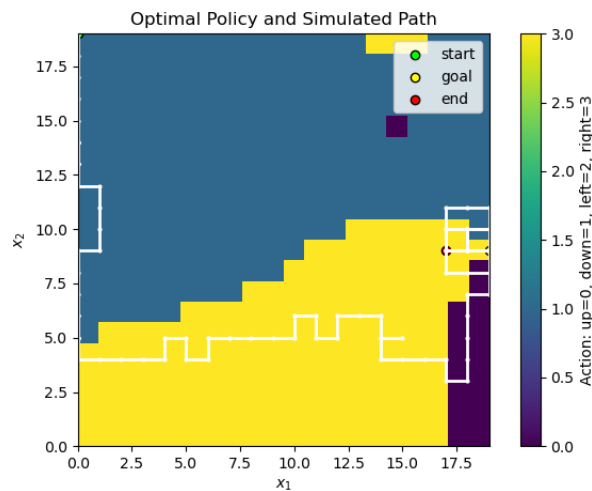


Figure 2: Optimal Policy Heatmap Plot

1.3 Code

Code in Appendix.

2 MPC feasibility

2.1 (a) Trajectories

Fig. 3 shows the closed and open loop trajectories for the system. The orange trajectory starts from the larger velocity initial condition, so it is harder to slow down because the input is bounded. Using $P = P_{\text{DARE}}$ gives a more useful terminal cost than $P = I$, since it approximates the infinite-horizon LQR cost-to-go after the short MPC horizon. This makes the predicted plans and the realized trajectory look more stabilizing and less short-sighted.

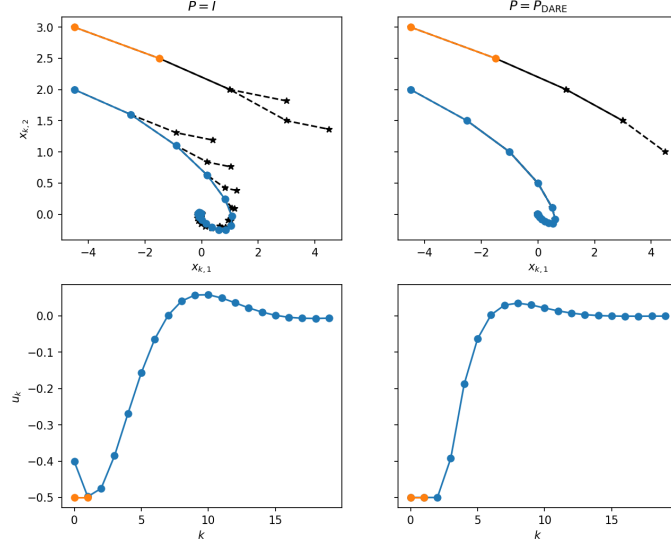


Figure 3: Trajectories

2.2 (b) Region of attraction

Fig. 4 shows the ROA obtained by simulating closed-loop MPC from each grid point and marking the states that converge to the origin. Longer horizons give a larger ROA because the controller can plan farther ahead. The strict terminal constraint $r_f = 0$ makes the ROA smaller, while $r_f = \infty$ leaves feasibility mostly limited by the state and input bounds.

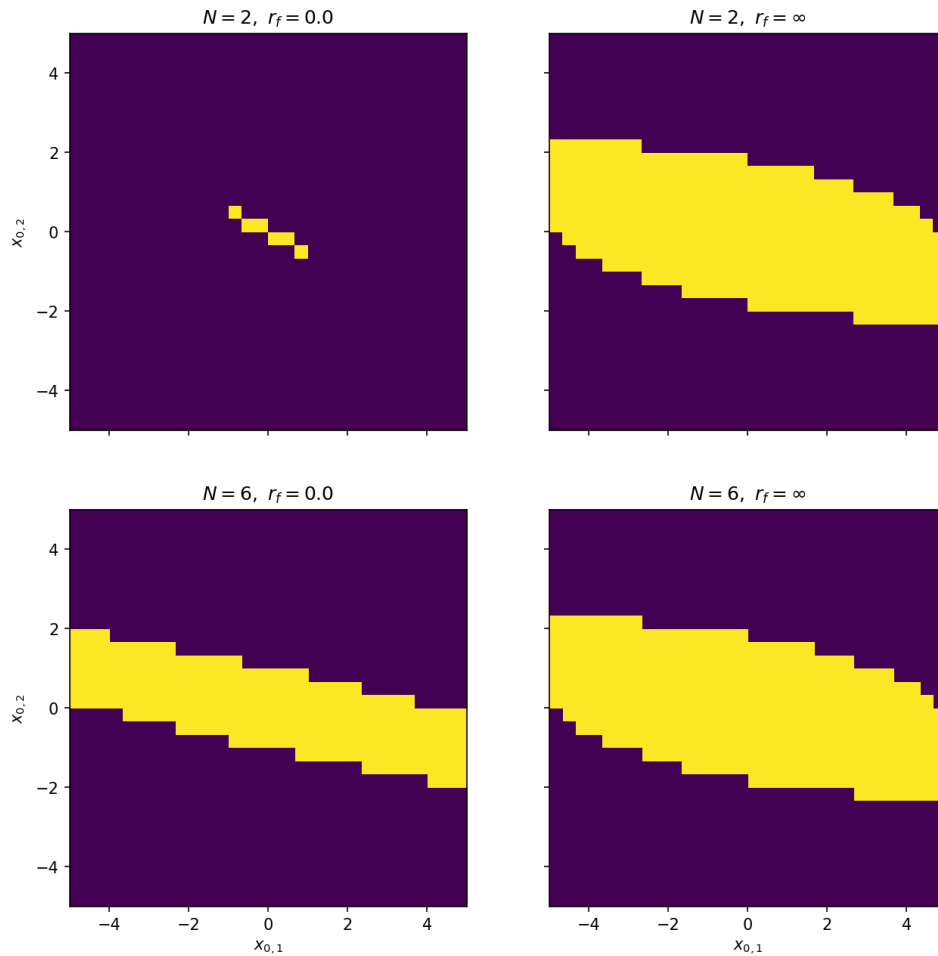


Figure 4: Regions of attraction for different horizon lengths and terminal constraints

2.3 Code

(a)

```

1  def do_mpc(
2  x0: np.ndarray,
3  A: np.ndarray,
4  B: np.ndarray,
5  P: np.ndarray,
6  Q: np.ndarray,
7  R: np.ndarray,
8  N: int,
9  rx: float,
10 ru: float,
11 rf: float,
12 ) -> tuple[np.ndarray, np.ndarray, str]:
13     """Solve the MPC problem starting at state 'x0'."""
14     n, m = Q.shape[0], R.shape[0]
15     x_cvx = cvx.Variable((N + 1, n))
16     u_cvx = cvx.Variable((N, m))
17
18     # PART (a): YOUR CODE BELOW #####
19     # INSTRUCTIONS: Construct and solve the MPC problem using CVXPY.
20
21     cost = 0.0

```

```

22 constraints = [x_cvx[0,:] == x0]
23
24 # N is the horizon
25 for t in range(N):
26     # running cost
27     cost += cvx.quad_form(x_cvx[t,:], Q) + cvx.quad_form(u_cvx[t,:], R)
28
29     constraints.append(x_cvx[t+1,:] == A@x_cvx[t,:] + B@u_cvx[t,:])
30     constraints.append(cvx.norm_inf(x_cvx[t,:]) <= rx)
31     constraints.append(cvx.norm_inf(u_cvx[t,:]) <= ru)
32
33 # auxiliary cost
34 cost += cvx.quad_form(x_cvx[N,:], P)
35 constraints.append(cvx.norm_inf(x_cvx[N,:]) <= rx)
36 constraints.append(cvx.norm_inf(x_cvx[N,:]) <= rf)
37
38 # END PART (a) #####
39
40 prob = cvx.Problem(cvx.Minimize(cost), constraints)
41 prob.solve(cvx.CLARABEL)
42 x = x_cvx.value
43 u = u_cvx.value
44 status = prob.status
45
46 return x, u, status

```

(b)

```

1 def compute_roa(
2     A: np.ndarray,
3     B: np.ndarray,
4     P: np.ndarray,
5     Q: np.ndarray,
6     R: np.ndarray,
7     N: int,
8     rx: float,
9     ru: float,
10    rf: float,
11    grid_dim: int = 21,
12    max_steps: int = 20,
13    tol: float = 1e-2,
14 ) -> np.ndarray:
15     """Compute a region of attraction."""
16     roa = np.zeros((grid_dim, grid_dim))
17     xs = np.linspace(-rx, rx, grid_dim)
18     for i, x1 in enumerate(xs):
19         for j, x2 in enumerate(xs):
20             x = np.array([x1, x2])
21             # PART (b): YOUR CODE BELOW #####
22             # INSTRUCTIONS: Simulate the closed-loop system for 'max_steps',
23             #                 stopping early only if the problem becomes
24             #                 infeasible or the state has converged close enough
25             #                 to the origin. If the state converges, flag the
26             #                 corresponding entry of 'roa' with a value of '1'.
27
28             for _ in range(max_steps):
29                 x_mpc, u_mpc, status = do_mpc(x, A, B, P, Q, R, N, rx, ru, rf)
30
31                 if status == "infeasible":
32                     break
33

```

```

34         if np.linalg.norm(x) <= tol:
35             roa[i, j] = 1
36             break
37
38         x = A @ x + B @ u_mpc[0, :]
39
40     if np.linalg.norm(x) <= tol:
41         roa[i, j] = 1
42
43     # END PART (b) #####
44 return roa

```

3 Terminal ingredients

(a)

Eigenvalues of A are $(0.9, 0.8)$, and lie in the unit circle, so A is asymptotically stable, and $u = 0$ is feasible as terminal controller and the terminal ingredients are designed for the autonomous system $x_{t+1} = Ax_t$. The terminal set X_T should satisfy $X_T \subseteq X$ and be a positive invariant for the prev. system:

$$x \in X_T \implies Ax \in X_T.$$

which guarantees recursive feasibility by shifting the MPC solution and appending $u = 0$.

$P \succ 0$ a symmetric, positive definite matrix

For stability, P must a symmetric, positive definite matrix ($P \succ 0$), from the Lyapunov equation.

(b)

For any $x \in \mathcal{X}_T$ (positive invariant set), $x^\top Wx \leq 1$.

Since $A^\top WA - W \preceq 0$, we get:

$$\begin{aligned} x^\top (A^\top WA - W)x &\leq 0 \\ (Ax)^\top WAx - x^\top Wx &\leq 0 \\ (Ax)^\top WAx &\leq x^\top Wx \leq 1 \end{aligned}$$

So $Ax_T \in \mathcal{X}_T$

For $I - r_x^2 W \preceq 0$:

$$\begin{aligned} x^\top (I - r_x^2 W)x &\leq 0 \\ x^\top x - x^\top r_x^2 Wx &\leq 0 \\ x^\top x &\leq x^\top r_x^2 Wx \\ x^\top x / r_x^2 &\leq x^\top Wx \leq 1 \\ x^\top x &\leq r_x^2 \end{aligned}$$

Which already complies with $x^\top x \leq \|x\|_2^2 \leq r_x^2$

(c)

First Constraint

Using the first hint

$$\begin{aligned} A^\top M^{-1}A - M^{-1} &\preceq 0 \\ M(A^\top M^{-1}A)M - MM^{-1}M &\preceq 0 \\ MA^\top M^{-1}AM - M &\preceq 0 \end{aligned}$$

Using Schur's complement lemma, the constraint is rewritten as:

$$\begin{bmatrix} M & AM \\ MA^T & M \end{bmatrix} \succeq 0$$

Second Constraint

$$\begin{aligned} I - r_x^2 M^{-1} &\preceq 0 \\ r_x^{-2} I &\preceq M^{-1} \end{aligned}$$

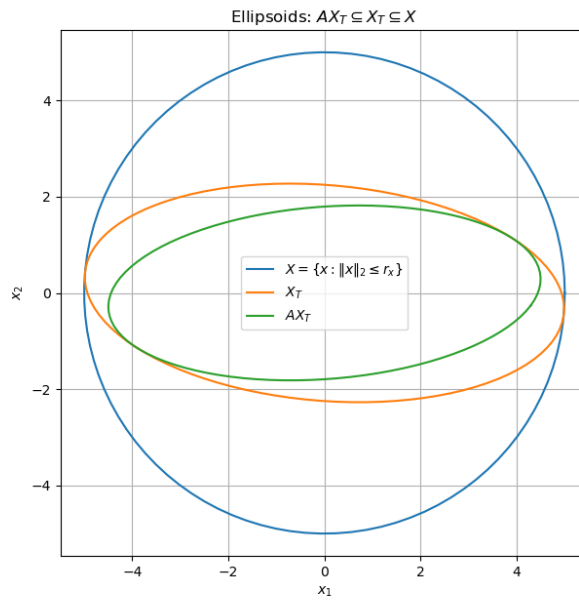
Using second hint

$$\begin{aligned} r_x^2 I &\succeq M \\ r_x^2 I - M &\succeq 0 \end{aligned}$$

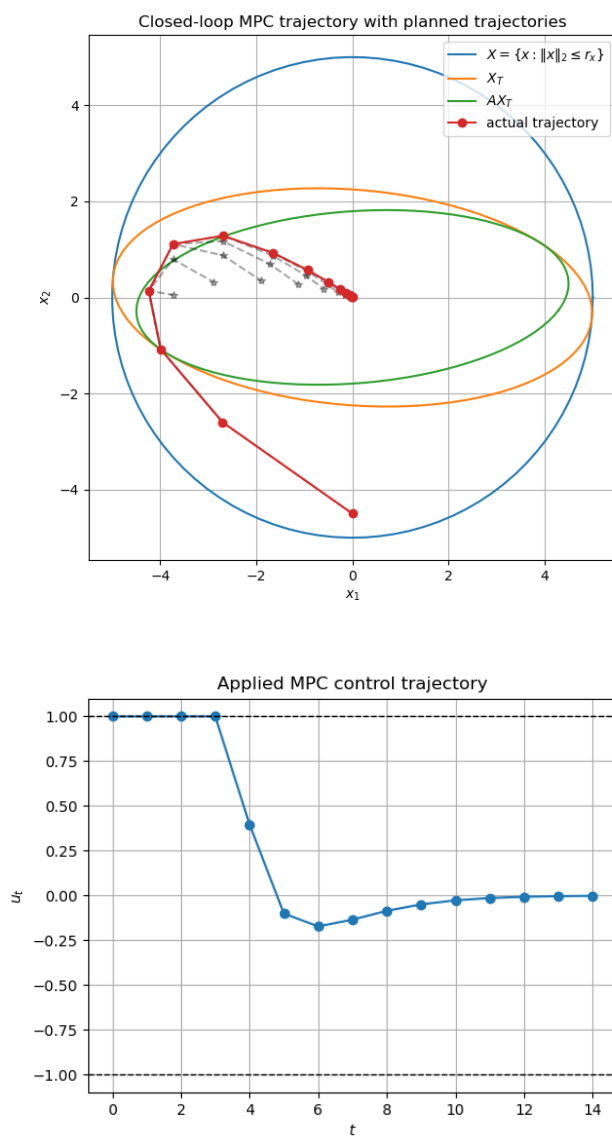
(d)

W=

$$\begin{bmatrix} 0.041 & 0.013 \\ 0.013 & 0.198 \end{bmatrix}$$



(d)



Code

Code in Appendix.

Code Appendix

Homework part	Code file
3.1	markovian_drone.py
3.4	terminal_ingredients.py

markovian_drone.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 ACTIONS = np.array([ [0,1], # up
5                      [0,-1], # down
6                      [-1,0], # left
7                      [1,0], # right
8                      ])
9
10 def idx_to_state(s,n=20):
11     x1 = s % n
12     x2 = s // n
13     return np.array([x1,x2])
14
15 def move(x,a,n=20):
16     x_next = x+a
17     x_next = np.clip(x_next,0,n-1)
18
19     return x_next[1]*n + x_next[0]
20
21 def value_iteration(n,eps=1e-6):
22     sigma = 10
23     gamma = 0.95
24     x_eye = np.array([15,15])
25     x_goal = np.array([19,9])
26     num_states = n*n
27     V = np.zeros(n*n)
28     opt_policy = np.zeros(n*n, dtype=int)
29
30     T = np.zeros((n*n,4,n*n)) # current, action, next
31     R = np.zeros(num_states)
32     R[x_goal[1] * n + x_goal[0]] = 1
33
34     for s in range(num_states):
35         # storm_influence
36         x = idx_to_state(s)
37         omega = np.exp(-np.linalg.norm(x - x_eye)**2 / (2 * sigma**2))
38
39         # Build transition
40         for a in range(4):
41             # specified action
42             next_s = move(x,ACTIONS[a])
43             T[s,a, next_s] += 1 - omega
44
45             # random action
46             for a_rand in range(4):
47                 next_s_rand = move(x,ACTIONS[a_rand])
48                 T[s,a,next_s_rand] += omega/4
49
50     diff = np.inf
51     while diff > eps:
52         V_new = np.zeros_like(V)
```

```

53
54     for s in range(num_states):
55         q = np.zeros(4)
56
57         for a in range(4):
58             q[a] = np.sum(T[s,a,:] * (R + gamma*V))
59
60         opt_policy[s] = np.argmax(q)
61         V_new[s] = q[opt_policy[s]]#np.max(q)
62
63     diff = np.max(np.abs(V_new - V))
64     V = V_new
65
66     return V, opt_policy
67
68 def simulate_MDP(policy,n):
69     sigma = 10
70     x_eye = np.array([15,15])
71     x_init = np.array([0,19])
72     time_steps = 100
73     path = [x_init.copy()]
74
75     x = x_init.copy()
76     for _ in range(time_steps):
77         s = x[1] * n + x[0]
78         action = policy[s]
79         omega = np.exp(-np.linalg.norm(x - x_eye)**2 / (2 * sigma**2))
80
81         if np.random.rand() < omega:
82             action = np.random.randint(4)
83
84         next_s = move(x,ACTIONS[action],n)
85         x = idx_to_state(next_s,n)
86         path.append(x.copy())
87
88     return np.array(path)
89
90 def plot_heatmaps(V,policy,path,n):
91     V_grid = V.reshape((n,n))
92     policy_grid = policy.reshape((n,n))
93
94     plt.figure()
95     plt.imshow(V_grid, origin='lower', extent=[0,n-1,0,n-1])
96     plt.colorbar(label='Value')
97     plt.xlabel('$x_1$')
98     plt.ylabel('$x_2$')
99     plt.title('Optimal Value Function')
100    plt.tight_layout()
101    # plt.show()
102    plt.savefig("optimal_value_heatmap.png")
103
104    plt.figure()
105    plt.imshow(policy_grid, origin='lower', extent=[0,n-1,0,n-1], vmin=0, vmax=3)
106    plt.colorbar(label='Action: up=0, down=1, left=2, right=3')
107    plt.plot(path[:,0], path[:,1], color='white', linewidth=2, marker='o', markersize=2)
108    plt.scatter(path[0,0], path[0,1], color='lime', edgecolor='black', label='start')
109    plt.scatter(19,9, color='yellow', edgecolor='black', label='goal')
110    plt.scatter(path[-1,0], path[-1,1], color='red', edgecolor='black', label='end')
111    plt.xlabel('$x_1$')
112    plt.ylabel('$x_2$')
113    plt.title('Optimal Policy and Simulated Path')
114    plt.legend()
115    plt.tight_layout()

```

```

116 plt.savefig("policy_heatmap_path.png")
117
118 def main():
119     n = 20
120
121     # Part A
122     V,policy = value_iteration(n)
123
124     # Part B
125     path = simulate_MDP(policy,n)
126
127     plot_heatmaps(V,policy,path,n)
128
129
130 if __name__ == '__main__':
131     main()

```

terminal.ingredients.py

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 import cvxpy as cp
5
6 def generate_ellipsoid_points(M, num_points=100):
7     """Generate points on a 2-D ellipsoid.
8     The ellipsoid is described by the equation
9      $\{x | x.T @ \text{inv}(M) @ x \leq 1\}$ ,
10    where 'inv(M)' denotes the inverse of the matrix argument 'M'.
11    The returned array has shape (num_points, 2).
12    """
13    L = np.linalg.cholesky(M)
14     $\theta$  = np.linspace(0, 2*np.pi, num_points)
15    u = np.column_stack([np.cos( $\theta$ ), np.sin( $\theta$ )]])
16    x = u @ L.T
17    return x
18
19 def semi_def_program():
20     A = np.array([
21         [0.9, 0.6],
22         [0.0, 0.8]
23     ])
24     B = np.array([
25         [0.0],
26         [1.0]
27     ])
28
29     rx = 5.0
30     n = A.shape[0]
31     M = cp.Variable((n, n), symmetric=True)
32
33     block = cp.bmat([
34         [M, A @ M],
35         [M @ A.T, M]
36     ])
37
38     constraints = [
39         M >> 0,
40         block >> 0,
41         rx**2 * np.eye(n) - M >> 0
42     ]

```

```

43
44 objective = cp.Maximize(cp.log_det(M))
45 prob = cp.Problem(objective, constraints)
46
47 prob.solve()
48
49 # print(M.value)
50 W = np.linalg.inv(M.value)
51 print("W=")
52 print(np.round(W, 3))
53
54 X_T = generate_ellipsoid_points(M.value)
55
56 X_T_next = X_T @ A.T
57
58 # X = {x | ||x||_2 <= rx}
59 theta = np.linspace(0, 2*np.pi, 100)
60 X_pts = np.column_stack([
61     rx * np.cos(theta),
62     rx * np.sin(theta)
63 ])
64
65 plt.figure(figsize=(7, 7))
66
67 plt.plot(X_pts[:, 0], X_pts[:, 1], label=r"$X=\{x: ||x||_2 \leq r_x\}$")
68 plt.plot(X_T[:, 0], X_T[:, 1], label=r"$X_T$")
69 plt.plot(X_T_next[:, 0], X_T_next[:, 1], label=r"$A \sqcup X_T$")
70
71 plt.axis("equal")
72 plt.grid(True)
73 plt.xlabel(r"$x_1$")
74 plt.ylabel(r"$x_2$")
75 plt.legend()
76 plt.title(r"Ellipsoids: $\sqcup A \sqcup X_T \sqcup \text{subseteq} X_T \sqcup \text{subseteq} X$")
77 plt.savefig("ellipsoids.png")
78
79 return A, B, M.value, W
80
81 def setup_mpc(A, B, W, N=4, rx=5.0, ru=1.0):
82     n, m = B.shape
83     Q = np.eye(n)
84     R = np.eye(m)
85     P = np.eye(n)
86
87     x0 = cp.Parameter(n)
88     x = cp.Variable((N + 1, n))
89     u = cp.Variable((N, m))
90
91     cost = 0.0
92     constraints = [x[0, :] == x0]
93
94     for t in range(N):
95         cost += cp.quad_form(x[t, :], Q) + cp.quad_form(u[t, :], R)
96         constraints += [
97             x[t + 1, :] == A @ x[t, :] + B @ u[t, :],
98             cp.norm(x[t, :], 2) <= rx,
99             cp.norm(u[t, :], 2) <= ru,
100         ]
101
102     cost += cp.quad_form(x[N, :], P)
103     constraints += [cp.quad_form(x[N, :], W) <= 1]
104
105     prob = cp.Problem(cp.Minimize(cost), constraints)

```

```

106     return prob, x0, x, u
107
108 def simulate_mpc(A, B, M, W, N=4, T=15, rx=5.0):
109     prob, x0_param, x_var, u_var = setup_mpc(A, B, W, N=N, rx=rx)
110
111     x = np.array([0.0, -4.5])
112     actual_states = [x.copy()]
113     planned_states = []
114     applied_controls = []
115
116     for _ in range(T):
117         x0_param.value = x
118         prob.solve()
119
120         x_plan = x_var.value
121         u_plan = u_var.value
122         planned_states.append(x_plan.copy())
123         applied_controls.append(u_plan[0, 0])
124
125         x = A @ x + B @ u_plan[0, :]
126         actual_states.append(x.copy())
127
128     actual_states = np.array(actual_states)
129     applied_controls = np.array(applied_controls)
130
131     plot_mpc_trajectories(A, M, actual_states, planned_states, rx)
132     plot_control_trajectory(applied_controls)
133
134 def plot_mpc_trajectories(A, M, actual_states, planned_states, rx):
135     X_T = generate_ellipsoid_points(M)
136     X_T_next = X_T @ A.T
137
138     theta = np.linspace(0, 2*np.pi, 100)
139     X_pts = np.column_stack([
140         rx * np.cos(theta),
141         rx * np.sin(theta)
142     ])
143
144     plt.figure(figsize=(7, 7))
145     plt.plot(X_pts[:, 0], X_pts[:, 1], label=r"$X=\{x:\|x\|_2\leq r_x\}$")
146     plt.plot(X_T[:, 0], X_T[:, 1], label=r"$X_T$")
147     plt.plot(X_T_next[:, 0], X_T_next[:, 1], label=r"$A_X_T$")
148
149     for x_plan in planned_states:
150         plt.plot(x_plan[:, 0], x_plan[:, 1], "--*", color="k", alpha=0.35)
151
152     plt.plot(actual_states[:, 0], actual_states[:, 1], "-o", label="actual_trajectory")
153     plt.axis("equal")
154     plt.grid(True)
155     plt.xlabel(r"$x_1$")
156     plt.ylabel(r"$x_2$")
157     plt.legend()
158     plt.title("Closed-loop MPC trajectory with planned trajectories")
159     plt.savefig("terminal_mpc_trajectory.png")
160
161 def plot_control_trajectory(applied_controls):
162     plt.figure()
163     plt.plot(np.arange(len(applied_controls)), applied_controls, "-o")
164     plt.axhline(1.0, color="k", linestyle="--", linewidth=1)
165     plt.axhline(-1.0, color="k", linestyle="--", linewidth=1)
166     plt.grid(True)
167     plt.xlabel(r"$t$")
168     plt.ylabel(r"$u_t$")

```

```
169     plt.title("Applied_MPC_control_trajectory")
170     plt.savefig("terminal_mpc_control.png")
171
172 def main():
173     A, B, M, W = semi_def_program()
174     simulate_mpc(A, B, M, W)
175
176 if __name__ == "__main__":
177     main()
```